Question 1 : Explain the fundamental differences between DDL, DML, and DQL commands in SQL. Provide one example for each type of command.

- DDL- Data Definition language. It changes the structure of the Data. Used to create, delete or modify database objects such as- Tables, Databases, views, indexes.

  Command: Create, Drop, alter, Truncate etc.

- DML- Data Manipulation language. It is used to manipulate the data inside the tables
  Commands- Insert, delete, update
- DQL- Data Query language. It is used to select the Data from the Database.
  Command: Select

Question 2 : What is the purpose of SQL constraints? Name and describe three common types of constraints, providing a simple scenario where each would be useful.

Ans: SQL constarints are used to maintain data integrity and consistency and it helps to prevent the invalid data being stored.

Three common type of consrtaints are-

Primary Key Constraints:

It uniquely identifies each row in a table.

Rules-

- Must be unique
- Can not contain null values
- One primary key in the table

```
CREATE TABLE Customers (
   Customer_ID INT PRIMARY KEY,
   Customer_Name VARCHAR(50)
);
```

So here primary key make sure that no two customer got the same customer id.

Foreign key Constarints:

A foreign key creates relationship between two tables.

A value in one table must exist in other table.

```
Create Table customer (
  Customer_id  PRIMARY KEY,
   Customer_name  VARCHAR(50)
);
```

```
create   table orders(
order_id int primary key,
customer_id int ,
foreign key(customer_id)
references customer(customer_id));
```

if the customer id is not in the customer table then that invalid customer_id will not be added to the order tables.

Question 3 : Explain the difference between LIMIT and OFFSET clauses in SQL. How would you use them together to retrieve the third page of results, assuming each page has 10 records?

Ans: Limit – how many rows you want to retrieve from the result.

Offset- How many rows to skip

```
# to select the only first 10 rows
select * from
customer
limit 10;

## to skip the first 10 rows
select    * from customer
offset 10;

## Now to get the 3rd page data when each page contains only  10 rows

select * from
customer
limit 30
offset    20;
```

Question 4 : What is a Common Table Expression (CTE) in SQL, and what are its main benefits? Provide a simple SQL example demonstrating its usage.

Ans: CTE- common table expression is a temporary table created in a single query to break the complex logic into smaller parts. This tables is created using the “WITH” keyword.

.

```
with spending as
(select customer_id, round(sum(amount),2) as Total_amount_spent
from Payment
group by Customer_id )

select  C.customer_id,  C.first_name,C.last_name, C.email, S.Total_amount_spent
from customer C Left join spending S
on C.customer_id= S.customer_id
order by Total_amount_spent desc
limit 5;
```

here you can see that a CTE “Spending “ is created first to where Total amount spend has been calculated after that in that same query using that ‘spending’ table we can get top 5 customer who spend the most.

Question 6 : Create a database named ECommerceDB and perform the following tasks:

```
create database ECommerceDB;
use EcommerceDB;

create table Categories (
CategoryID INT PRIMARY KEY,
CategoryName VARCHAR(50) NOT NULL UNIQUE
```

```
);

Create   table Products (
ProductID INT PRIMARY KEY,
ProductName VARCHAR(100) NOT NULL UNIQUE,
CategoryID INT ,
foreign key (CategoryID)
references Categories(CategoryID),
Price DECIMAL(10,2) NOT NULL,
StockQuantity INT);



Create table  Customers (
CustomerID INT PRIMARY KEY,
CustomerName VARCHAR(100) NOT NULL,
Email VARCHAR(100) UNIQUE,
JoinDate DATE );


create table Orders (
OrderID INT PRIMARY KEY,
CustomerID INT,
foreign key (CustomerID)
references Customers(CustomerID),
OrderDate DATE NOT NULL,
TotalAmount DECIMAL(10,2));


## Insert valuse in table customers
```

```
insert into Categories (CategoryID, CategoryName)
values
     (1, 'Electronics'),
    (2, 'Books'),
    (3, 'Home Goods'),
    (4, 'Apparel');

select  * from Categories;

insert into          Products values
 (101, 'Laptop Pro', 1, 1200.00, 50),
    (102, 'SQL Handbook', 2, 45.50, 200),
    (103, 'Smart Speaker', 1, 99.99, 150),
    (104, 'Coffee Maker', 3, 75.00, 80),
    (105, 'Novel: The Great SQL', 2, 25.00, 120),
    (106, 'Wireless Earbuds', 1, 150.00, 100),
    (107, 'Blender X', 3, 120.00, 60),
    (108, 'T-Shirt Casual', 4, 20.00, 300);

select *from Products;



insert into Customers
values (1, 'Alice Wonderland', 'alice@example.com', '2023-01-10'),
    (2, 'Bob the Builder', 'bob@example.com', '2022-11-25'),
    (3, 'Charlie Chaplin', 'charlie@example.com', '2023-03-01'),
    (4, 'Diana Prince', 'diana@example.com', '2021-04-26');

select * from Customers;
```

```
insert into Orders
values
(1001, 1, '2023-04-26', 1245.50),
   (1002, 2, '2023-10-12', 99.99),
   (1003, 1, '2023-07-01', 145.00),
   (1004, 3, '2023-01-14', 150.00),
   (1005, 2, '2023-09-24', 120.00),
   (1006, 1, '2023-06-19', 20.00);

select * from Orders;
```

Question 7 : Generate a report showing CustomerName, Email, and the TotalNumberofOrders for each customer. Include customers who have not placed any orders, in which case their TotalNumberofOrders should be 0. Order the resultsby CustomerName.

```
select   CustomerName, Email , count(OrderID) as TotalNumberofOrders
from Customers Cu Left join Orders O
on         Cu.CustomerID= O.CustomerID
group by CustomerName , Email
order by CustomerName asc;
```

Question 8 : Retrieve Product Information with Category: Write a SQL query to display the ProductName, Price, StockQuantity, and CategoryName for all products. Order the results by CategoryName and then ProductName alphabetically.

```
select ProductName, Price, StockQuantity, CategoryName
```

```
from products P Left join Categories C
on P.CategoryID = C.CategoryID
order by CategoryName , ProductName;
```

Question 9 : Write a SQL query that uses a Common Table Expression (CTE) and a Window Function (specifically ROW_NUMBER() or RANK()) to display the CategoryName, ProductName, and Price for the top 2 most expensive products in each CategoryName.

```
With RankedProduct as (
select ProductName, CategoryName, Price,
RANK() OVER (partition by CategoryName ORDER BY Price DESC) AS PriceRank
from Products P left join Categories C
on P.CategoryID= C.CategoryID )
select Pricerank, ProductName, CategoryName, Price
from RankedProduct
where Pricerank <=2;
```

Question 10 : You are hired as a data analyst by Sakila Video Rentals, a global movie rental company. The management team is looking to improve decision-making by analyzing existing customer, rental, and inventory data.

```
use sakila;
```

1.Identify the top 5 customers based on the total amount they've spent. Include customer name, email, and total amount spent.

```
with spending as
(select customer_id, round(sum(amount),2) as Total_amount_spent
from Payment
```

```
group by Customer_id )

select  C.customer_id,  C.first_name,C.last_name, C.email, S.Total_amount_spent
from customer C Left join spending S
on C.customer_id= S.customer_id
order by Total_amount_spent desc
limit 5;
```

Which 3 movie categories have the highest rental counts? Display the category name and number of times movies from that category were rented.

```
with Rental_category as (
select  C.name as Category_name, FC.film_id , F.rental_duration
from
film_category FC left join Category C
on FC.category_id=C.category_id
left join film F
on FC.film_id=F.film_id)

select     Category_name ,  sum(rental_duration) as Rent_times
from Rental_category
group by Category_name
order by Rent_times desc
limit 3;
```

Show the total revenue per month for the year 2023 to analyze business seasonality.

```
Select   date_format(payment_date,'%M') as Month, sum(amount) as Revenue
from payment
group by month;
```

#Identify customers who have rented more than 10 times in the last 6 months.

```
select * from rental;

select  rental_id, month(rental_date) as Month, customer_id
from rental
where month(rental_date)>=6;

select count(customer_id) as Rent_times , customer_id
from rental
where rental_date >='2005-06-01'
group by customer_id
;
```